

Assignment: CI/CD for Containerized Microservices

Theme

CI is not a ceremonial badge.

A green checkmark is only useful if it proves something important. For this assignment, CI must prove that the application code works, the services build successfully, the containers can be packaged, and the exact image version can be deployed later.

You are extending the previous containerization lab. Each group must now automate quality checks, Docker image builds, image publishing, and deployment instructions using GitHub Actions.

The expected project shape is similar to the Nginx gateway microservices lab:

```
.
├── .github/workflows/container-ci-cd.yml
├── services/
│   ├── service-a/Dockerfile
│   ├── service-b/Dockerfile
│   └── service-c/Dockerfile
├── nginx/
│   └── nginx-docker.conf
├── docker-compose.yml
├── docker-compose.prod.yml
├── scripts/deploy.sh
├── .dockerignore
├── .env.example
└── README.md
```

The architecture expectation remains:

```
client -> nginx gateway -> service-a -> service-b/service-c
```

Only Nginx should expose a host port. Internal services should communicate through Docker networks.

Learning Outcomes

By the end of the assignment, each group should be able to:

1. Build a meaningful GitHub Actions CI pipeline.
 2. Run application dependency installation, tests, and build checks before Docker packaging.
 3. Build Docker images from validated code.
 4. Push versioned images to Docker Hub.
 5. Deploy using commit-pinned image tags instead of `latest`.
 6. Apply Dockerfile and Compose hardening feedback from peer review.
 7. Peer review another group's deployed artifacts for traceability, pinning, and runtime safety.
-

Part 1: Pull Request CI

Every pull request to `main` must run a verification pipeline.

Create:

```
.github/workflows/container-ci-cd.yml
```

Minimum PR workflow:

```
name: Container CI/CD

on:
```

```
pull_request:
  branches: [main]
push:
  branches: [main]
workflow_dispatch:

permissions:
  contents: read

jobs:
  verify:
    name: Verify code, build, and containers
    runs-on: ubuntu-latest

    strategy:
      matrix:
        include:
          - service: service-a
            path: ./services/service-a
          - service: service-b
            path: ./services/service-b
          - service: service-c
            path: ./services/service-c

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
          cache-dependency-path: ${matrix.path}/package-lock.json

      - name: Install dependencies
        working-directory: ${matrix.path}
        run: npm ci

      - name: Run tests
        working-directory: ${matrix.path}
        run: npm test
```

```
- name: Build application
  working-directory: ${ matrix.path }
  run: npm run build --if-present

- name: Build Docker image locally
  run: |
    docker build \
      -t ${ matrix.service }:${ github.sha } \
      -f ${ matrix.path }/Dockerfile \
      ${ matrix.path }
```

This PR workflow must not push images to Docker Hub. Pull requests should verify quality, not publish deployable images.

A pull request should not be merged if:

- npm ci fails
- tests fail
- application build fails
- Docker image build fails
- Docker Compose config is invalid

Part 2: Compose-Level Verification

After individual services build, the workflow must validate the full container system.

Add a second job:

```
verify-compose:
  name: Verify Docker Compose stack
  runs-on: ubuntu-latest
  needs: verify
```

```
steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Validate Compose file
    run: docker compose config

  - name: Build Compose stack
    run: docker compose build --pull

  - name: Start Compose stack
    run: docker compose up -d

  - name: Show running services
    run: docker compose ps

  - name: Check gateway health
    run: |
      sleep 10
      curl --fail http://localhost:8080/service-a/health

  - name: Stop Compose stack
    if: always()
    run: docker compose down -v
```

The gateway health check should match your actual route. If your route differs, document it in the README.

Part 3: Push Images on Main Only

Only successful pushes to `main` should publish images to Docker Hub.

The pipeline must push images using commit-hash tags.

Required GitHub Repository Variable:

DOCKERHUB_USERNAME

Required GitHub Repository Secret:

DOCKERHUB_TOKEN

Publishing job:

```
publish:
  name: Publish Docker images
  runs-on: ubuntu-latest
  needs: verify-compose
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'

  strategy:
    matrix:
      include:
        - service: service-a
          path: ./services/service-a
        - service: service-b
          path: ./services/service-b
        - service: service-c
          path: ./services/service-c

  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Login to Docker Hub
      uses: docker/login-action@v3
      with:
        username: ${vars.DOCKERHUB_USERNAME}
        password: ${secrets.DOCKERHUB_TOKEN}

    - name: Set short commit hash
      run: echo "SHORT_SHA=${GITHUB_SHA::7}" >> "$GITHUB_ENV"
```

```

- name: Build and push image
  uses: docker/build-push-action@v6
  with:
    context: ${ matrix.path }
    file: ${ matrix.path }/Dockerfile
    push: true
    tags: |
      ${ vars.DOCKERHUB_USERNAME }/${
        github.event.repository.name }-${ matrix.service }:sha-${
        env.SHORT_SHA }
    labels: |
      org.opencontainers.image.revision=${ github.sha }
      org.opencontainers.image.source=${ github.server_url }/${
        github.repository }

- name: Write deployment summary
  run: |
    echo "### Published Image" >> "$GITHUB_STEP_SUMMARY"
    echo "Service: ${ matrix.service }" >> "$GITHUB_STEP_SUMMARY"
    echo "Image: ${ vars.DOCKERHUB_USERNAME }/${
      github.event.repository.name }-${ matrix.service }:sha-${SHORT_SHA}" >>
      "$GITHUB_STEP_SUMMARY"

```

Allowed:

```
group/repo-service-a:sha-a1b2c3d
```

Not allowed for deployment:

```
group/repo-service-a:latest
group/repo-service-a:main
group/repo-service-a:dev
```

Part 4: Production Compose File

Create:

```
docker-compose.prod.yml
```

This file must use pushed Docker Hub images.

It must not build locally.

Good:

```
services:
  service-a:
    image: ${DOCKERHUB_USERNAME}/${APP_NAME}-service-a:${IMAGE_TAG}
```

Bad:

```
services:
  service-a:
    build: ./services/service-a
```

Expected production pattern:

```
services:
  service-a:
    image: ${DOCKERHUB_USERNAME}/${APP_NAME}-service-a:${IMAGE_TAG}
    environment:
      BIND_HOST: 0.0.0.0
      SERVICE_B_URL: http://service-b:3002
      SERVICE_C_URL: http://service-c:3003
    depends_on:
```



```

- service-b
- service-c
networks:
- frontend
- backend
restart: unless-stopped

service-b:
image: ${DOCKERHUB_USERNAME}/${APP_NAME}-service-b:${IMAGE_TAG}
environment:
  BIND_HOST: 0.0.0.0
  SERVICE_C_URL: http://service-c:3003
networks:
- backend
restart: unless-stopped

service-c:
image: ${DOCKERHUB_USERNAME}/${APP_NAME}-service-c:${IMAGE_TAG}
environment:
  BIND_HOST: 0.0.0.0
networks:
- backend
restart: unless-stopped

nginx:
image: nginx:1.27-alpine
ports:
- "8080:80"
volumes:
- ./nginx/nginx-docker.conf:/etc/nginx/conf.d/default.conf:ro
depends_on:
- service-a
networks:
- frontend
restart: unless-stopped

networks:
frontend:
backend:
  internal: true

```

Only Nginx should expose a host port.

Part 5: Deployment Script

Create:

```
scripts/deploy.sh
```

Required behavior:

```
#!/usr/bin/env bash
set -euo pipefail

IMAGE_TAG="${1:-}"

if [ -z "$IMAGE_TAG" ]; then
    echo "Usage: ./scripts/deploy.sh sha-<short-commit-hash>"
    echo "Example: ./scripts/deploy.sh sha-a1b2c3d"
    exit 1
fi

export IMAGE_TAG
export APP_NAME="${APP_NAME:-$(basename "$PWD")}"

if [ -z "${DOCKERHUB_USERNAME:-}" ]; then
    echo "Missing DOCKERHUB_USERNAME"
    exit 1
fi

echo "Deploying ${APP_NAME} using image tag: ${IMAGE_TAG}"

docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d --remove-orphans
docker compose -f docker-compose.prod.yml ps
```

Run it like this:

```
export DOCKERHUB_USERNAME=<dockerhub-username>
export APP_NAME=<repo-name>

./scripts/deploy.sh sha-a1b2c3d
```

The script must not use `latest`.

Part 6: Dockerfile Expectations

Every service Dockerfile must show improvement from the previous peer review.

Minimum expectations:

- No FROM image uses `:latest`
- Base images are version-pinned
- `npm ci` is used instead of `npm install`
- `package-lock.json` is respected
- App runs as non-root where possible
- No secrets are copied into the image
- `.dockerignore` is present and useful
- Build and runtime concerns are separated where appropriate
- EXPOSE documents the intended service port

Example direction:

```
FROM node:20.18.1-alpine

WORKDIR /app

COPY package*.json ./
```

```
RUN npm ci --omit=dev
```

```
COPY . .
```

```
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
```

```
USER appuser
```

```
EXPOSE 3001
```

```
CMD ["node", "server.js"]
```

Stronger groups should use multi-stage builds where their application has a build step.

Part 7: README Update

Add this section:

```
## Container CI/CD Deployment
```

```
### Latest deployed version
```

```
Commit:
```

```
`<full-commit-hash>`
```

```
Image tag:
```

```
`sha-<short-commit-hash>`
```

```
Images:
```

- ``<dockerhub-username>/<repo-name>-service-a:sha-<short-commit-hash>``
- ``<dockerhub-username>/<repo-name>-service-b:sha-<short-commit-hash>``
- ``<dockerhub-username>/<repo-name>-service-c:sha-<short-commit-hash>``

```
### Deploy
```

```
```bash
cp .env.example .env
export DOCKERHUB_USERNAME=<dockerhub-username>
export APP_NAME=<repo-name>
./scripts/deploy.sh sha-<short-commit-hash>
```

## Verify

```
docker compose -f docker-compose.prod.yml ps
curl http://localhost:8080/service-a/health
```

---

# Acceptance Criteria

## A. Pull Request CI

```text

- [] Pull requests to main trigger CI
- [] npm ci runs successfully for each service
- [] Tests run for each service
- [] Build command runs where applicable
- [] Docker image builds locally for each service
- [] Failed tests or failed builds block merge

B. Compose Verification

- [] docker compose config passes
- [] docker compose build --pull succeeds
- [] docker compose up -d starts the stack
- [] Gateway health check passes
- [] Stack is cleaned up after CI

C. Image Publishing

- [] Images are pushed only after merge/push to main
- [] Images are pushed to Docker Hub
- [] Images use sha-<short-commit-hash> tags
- [] Deployment docs do not use latest
- [] Image labels include source repo and revision

D. Secrets and Environment Handling

- [] DOCKERHUB_TOKEN is stored as a GitHub Secret
- [] DOCKERHUB_USERNAME is stored as a GitHub Variable or documented clearly
- [] No real .env file is committed
- [] No API keys, passwords, tokens, or secrets are committed
- [] .env.example documents required runtime variables

E. Dockerfile Quality

- [] No Dockerfile uses FROM image:latest
- [] Base images are version-pinned
- [] npm ci is used
- [] App containers avoid root where possible
- [] .dockerignore excludes node_modules, .git, logs, cache files, and env files
- [] Docker image does not include unnecessary build files

F. Production Compose Readiness

- [] docker-compose.prod.yml exists

- [] Production Compose uses image: not build:
- [] IMAGE_TAG controls deployed version
- [] Only nginx exposes a host port
- [] Internal services do not expose host ports
- [] Backend network is separated
- [] Backend network is internal where appropriate
- [] Restart policies are configured

G. Deployment Script

- [] scripts/deploy.sh exists
- [] Script accepts sha-<commit> as an argument
- [] Script fails clearly when tag is missing
- [] Script pulls images before starting services
- [] Script uses docker-compose.prod.yml
- [] Script does not hard-code latest

Saturday Peer Review

Saturday is a deployment review gate, not a demo ceremony.

Each group must review another group's repository, GitHub Actions run, Docker Hub images, Dockerfiles, Compose files, and deployment script.

Reviewers must check:

- [] CI proves code quality before Docker packaging
- [] Tests are not skipped
- [] Failed tests would block merge
- [] Images are commit-tagged
- [] Images are pullable from Docker Hub
- [] Runtime Compose does not rebuild locally

```
[ ] Containers avoid root where possible
[ ] No secrets are committed
[ ] Only Nginx exposes a public port
[ ] Internal services are network-isolated
[ ] Deployment works using ./scripts/deploy.sh sha-<hash>
```

Useful commands:

```
docker pull <dockerhub-user>/<repo>-service-a:sha-<hash>
docker image inspect <dockerhub-user>/<repo>-service-a:sha-<hash>
docker compose -f docker-compose.prod.yml config
docker compose -f docker-compose.prod.yml up -d
docker compose -f docker-compose.prod.yml ps
curl http://localhost:8080/service-a/health
```

Submission Requirements

Each group must submit:

1. GitHub repository link
2. Pull request showing CI validation
3. Link to successful GitHub Actions run on main
4. Docker Hub image names
5. Commit hash used for deployment
6. README deployment section
7. Saturday peer-review report

Minimum Passing Standard

A group cannot pass unless all of the following are true:

- ☐ CI runs tests before Docker image build
- ☐ CI builds application code before Docker image build
- ☐ Docker images are pushed to Docker Hub
- ☐ Images use sha-`<commit>` tags
- ☐ No real secrets are committed
- ☐ Production Compose pulls images instead of building locally
- ☐ Deployment script runs a specific image version
- ☐ Saturday peer review is completed

A green pipeline that does not test, build, package, and trace the artifact is not CI/CD. It is decoration.